Machine Learning Tek5 2026-2027 project

NICOLAS LE HIR
nicolas.lehir@epitech.eu

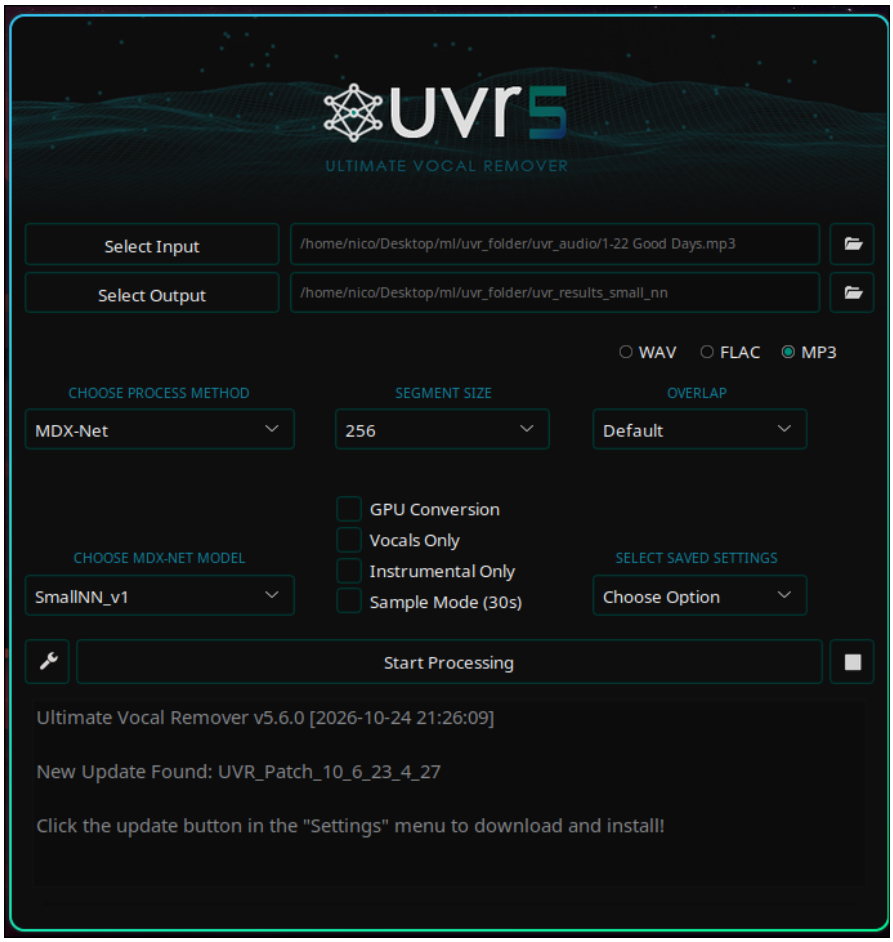


FIGURE 1 – UVR GUI

TABLE DES MATIÈRES

1 Introduction 2
1.1 Organization . . . 2
1.1.1 Course material . . . 2
1.2 Defense . . . 2
1.2.1 Repository . . . 3
1.2.2 Usage of AI . . . 3
1.2.3 Guidelines for the defense . . . 3
2 Tasks 4
2.1 General ML knowledge . . . 4
2.2 Training time optimization on MNIST . . . 4
2.3 Training and fine tuning for vocal removal . . . 5
2.3.1 Benchmark model . . . 5
2.3.2 Training a small neural network from scratch . . . 6
2.3.3 Fine tuning a pre-trained model on a degraded audio . . . 6
2.3.4 Scoring and testing . . . 6
2.4 Compression of a model for Food 101 classification . . . 6
2.5 Finding the best model on a tabular dataset . . . 7
2.6 Clustering on the Fashion MNIST dataset . . . 7
A Ultimate Vocal Remover (UVR) in more details 7
A.1 Installation . . . 7
A.2 GUI and usage. . . . 8
A.3 Models . . . 8
A.4 Digital audio, inputs and outputs of the MDX-Net models (chunks and spectrograms) . . . 8
A.5 Model data and json files . . . 10
A.6 MUSDB18 dataset . . . 11

1 INTRODUCTION

In this project we explore various branches of machine learning, in an attempt to give an partial overview of the domain. The studied datasets are chosen in order to provide interesting problems, while at the same time not requiring a computing power that would be forbidding for general use hardware.

You are encouraged to explore and test various approaches, found both by researching literature and proposing original, personal methods.

1.1 Organization

1.1.1 Course material

You can find the course material at this address :
`github.com/nlehir/ML_tek5/`

1.2 Defense

There will be a final defense to present your work. Please make a couple of slides per task, however there is no minimum number of slides (if only 1 or 2 slides are enough for you, it is fine). During the defense, please respect the following maximum durations :

— 2.1 (General ML knowledge) (5 min)

— 2.2 (MNIST fast training) (7 min)

— 2.3 (UVR Vocal Removal) (9 min)
— 2.4 (Model compression on Food101) (7 min)
— 2.5 (Tabular data)(6 min)
— 2.6 (Fashion-MNIST clustering) (6 min)

1.2.1 Repository

The project consists in several independent exercises. **Before the defense, please send me your repo url**, so that I can browse the code. You can push your work to the Epitech repo dedicated to the project, inside the course team. Please do not send a fork of the course repo, with some added files. Instead, make a repo containing only your project files.

1.2.2 Usage of AI

As this course is not a production course, I encourage you to take it as an opportunity to understand what you do and learn things, whatever the percentage of your code is written by AI.

1.2.3 Guidelines for the defense

All processing should be made with python3. Please include only **only technical** information in the defense.

During the defense, what is expected is that you explain your process to arrive to the results. Unsuccessful trials and methods are welcome ! It is very important to draw **conclusions** for each task. For each task, please include some short docstrings (if using scripts) or markdown (if using notebooks) that make it easier to browse the code.

— **mandatory :** for each task (except for 2.1) :
  — quickly define the objective with an exact and minimalistic ML vocabulary.
  — describe what the inputs and outputs of the model are. Recall the number of features and of samples, briefly describe the features.
— provide an **evaluation** or multiple evaluations of the obtained estimator(s), thanks to scorings of your choice. Be **quantitative** and precise about the results. When you mention the score of an estimator, **always** explicitely mention whether it is a train, test, validation, or cross-validation score (see the "scoring" chapter in the class).
— discuss the results obtained. To what extent have we solved the problem ? Could the model be actually used ? What precision can be expected with the model on unseen data ?
— Example of explanations and discussions that are expected in the defense : discussion over the choice of models (model selection), hyperparameters, details on the optimization (or model training) process, and conclusions on which method(s) worked best, or worst, or over the choice of preprocessing methods on the datasets.
— Example of explanations **not** to include in the defense :
  — high level presentation of the course itsself and of the tasks : please stay technical.
  — presentation of elementary python function, or library functions from torch, numpy, matplotlib, or the libraries themselves. You should **not** present these. You can however discuss specific functions or methods that are important for the optimization process, for instance in pytorch

or tensorflow algorithms and optimizers (but again, you do not need to explain what pytorch is).

— never forget to label the axes of plots

— never forget to mention the dimensions (unités) of the quantities

— if relevant or necessary, preprocess the data, and to justify this preprocessing. You could compare the estimator(s) obtained with and without preprocessing.

2 TASKS

2.1 General ML knowledge

Define these concepts in your own words, and illustrate them with examples :

— feature, label

— overfitting

— gradient and gradient descent

— hyperparameter

— the difference between supervised and unsupervised learning

Why is gradient descent usually not performed on hyperparameters ?

2.2 Training time optimization on MNIST

As we have seen during the course, we can reach more than 0.99 of test accuracy in a couple of minutes on a laptop on the MNIST dataset (https://en.wikipedia.org/wiki/MNIST_database). However, in many machine learning applications, especially in embedded systems, there is a need to optimize the learning time and also the inference time.

Look for the **fastest possible way** to reach a 0.97 test accuracy, on a given hardware of your choice. The hardware that you use may be your laptop (you can also compare two laptops). You need to briefly present the hardware used (type of processor, frequency, number of cores, etc)

In this exercise, what we measure is the complete learning procedure : data loading + optimization time. We would like to **accelerate** it. Please also monitor the impact of the various methods on the **inference time**.

Super important : Please note that your objective is **not to simply reach the target test accuracy**. Your objective is to compare different methods to **reach this accuracy as fast as possible**. You can for instance isolate one parameter of the learning process, for example :

— the learning rate, and plot the time to reach the target accuracy as a function of the learning rate.

— the numerical precision and / or type of the numbers used at various stages of the pipeline

— these are just 2 possible examples amongst many possibilities. Please come up with other variables to explore !

Profile learning and inference (for instance with torch profilers and trace visualization with perfetto) and explore any acceleration method available. Provide **visual representations** of the trade-offs and of the impact of the various parameters. You can for instance isolate one parameter of the learning process, like the learning rate, and plot the time to reach the target accuracy as a function of the learning rate. But this is just one possible example amongst many possibilities.

Note that you can stop model training as soon as the target accuracy is reached.

It is required that you explore at least one method taken from a book or from a scientific article. You can find the book or the article yourself, or use one of the references presented during the course. For instance, part II of the Deep learning book (https://www.deeplearningbook.org/) contains many interesting discussions, but other sources are accepted as well. Any research is welcome, including research on libraries that are not as famous as pytorch and tensforflow, like jax.

Here are some suggestions :

— you can start from the architecture that we use in this example : https://github.com/nlehir/ML_tek5/tree/main/code/day_3/2_neural_networks/mnist and then try other methods to accelerate learning or inference time.

— try to **isolate** parameters by monitoring the change in speed after changing only one aspect of the pipeline, leaving the rest similar.

You are encouraged to present and analyse results of code profiling in order to find speed bottlenecks. Pay attention to the fact that profiling GPUs or pytorch code might have specificities.

2.3 Training and fine tuning for vocal removal

Ultimate vocal remover (https://ultimatevocalremover.com/, https://github.com/Anjok07/ultimatevocalremovergui/releases/tag/v5.6) allows to separate the stems from an audio music file (this is sometimes useful in the music production world). In this context, the word "stem" stands for a main part in a song like the vocals, the drums, the bass, the synthesisers, etc.

The GUI allows to choose between various available pretrained models families, among which we will use the MDX-Net one. More details about UVR and its models are given in Appendix A.

We note that the UVR contains the **inference code**, but not the **training** code. In this task, we will explore training our own models and compare their output to the models that are already available in UVR.

To get familiar with UVR, start by following sections A.1 and A.2 of the Appendix.

2.3.1 Benchmark model

We will first add a very simple benchmark model to the GUI, without even training a model. The result will be the "mono" signal, which will yield a poor quality of vocal removal. The objective here is to verify that we can manually add a model, and to get more familiar with the audio concepts and vocabulary. Follow these steps :

— copy the **project/uvr_my_models/** folder to the uvr repo root. This folder contains :

  — **0_download_data.py** : useful for 2.3.2 and/or 2.3.4 : fetches a small version of the MUSDB18 audio dataset (https://zenodo.org/record/3270814/files/MUSDB18-7-STEMS.zip). The audio is reduced to 7 seconds compared to the original dataset, which you are encouraged to explore.

  — **1_create_benchmark_model.py** : defines the mono channel benchmark model, and exports it to the .onnx format, used by the MDX-Net models in UVR.

  — **5_degrade_data.py** : creates a new, degraded audio dataset from the one fetched by **0_download_data.py**. Useful for 2.3.4.

  — **src/data_preparation.py** : utility functions to parse audio clips to (please find more information in the Appendix A and in the file comments). Useful in all steps.

  - **src/degradation.py** : defines utility functions to degrade audio, useful in 2.3.4.
  - **src/model_saving.py** : defines utility functions to export the model and its metadata (please find more information in the Appendix A and in the file comments). Useful in all steps.
  - **src/spectograms.py** : defines **stft()**, a function that returns spectrograms from audio chunks. Useful in all steps.
- run **1_create_benchmark_model.py**. This will export an .onnx model to **models/MDX_Net_Models/Benchmark_CentreChannel.onnx**. This model will now be available in the UVR GUI.

This step is also for you to familiarize yourself with the inputs that are received by the MDX-Net models : **spectograms** of audio pieces called **chunks**. These concepts are defined more extensively in A.4. For all these topics, please find more information in the Appendix A and in the file comments

2.3.2 Training a small neural network from scratch

The problem of audio stem separation is not an easy one. We can not expect to easily train a neural network, with a random architecture, and have good results without a significant amount of trial and error.

In most industrial applications, people usually start from a pretrained architecture, and fine tune from there (meaning, they retrain the model, using the pretrained one as an initialisation), using their own particular dataset.

Still, in this step, we will try to see which results we can get by training a neural network from scratch, to witness this difficulty directly. Train a small neural network (a small number of layers at most) add it to the GUI and compare the results to the simple benchmark. You should try several datasets, depending on you hardware constraints. An example, small dataset can be fetch using **0_download_data.py**. You may experiment with data augmentation methods.

2.3.3 Fine tuning a pre-trained model on a degraded audio

We will now try to improve the quality of vocal removal, on degraded audio. By running **5_degrade _data.py**, you can produce an example degraded audio dataset, but you should experiment with difference and/or other datasets. Using these datasets, **fine-tune** a MDX-Net model of your choice in an attempt to improve it on degraded audio. Experiment with anything : the source dataset, the degradation method, the hyperparameters, data-augmentation methods, the pre-trained model, etc.

2.3.4 Scoring and testing

For steps 2.3.1, 2.3.2 and 2.3.4 :

- compute a **numerical evaluation/scoring** of the model. A possible score here is the Signal to Distortion ratio (SDR).
- produce audio examples in order to compare the different approaches to the original UVR

2.4 Compression of a model for Food 101 classification

The Food 101 dataset (`https://huggingface.co/datasets/ethz/food101`) is a classical classification dataset. Many models have already been trained in order to solve this problem. You objective is to find the smallest, lightest possible neural network model that keeps a score close to the networks that achieve optimal performance. Here, we are interested in **compressing** an already trained network, in order

to **decrease the size in memory, number of parameters, and inference time**. To do so, many methods exist, such a quantization, distillation and other methods that you should explore. You are encouraged to read research articles and use methods extracted from those. Study the trade-off between the amount of compression and the final test score, and illustrate this trade-off with visualizations.

2.5 Finding the best model on a tabular dataset

Solve the problem of predicting the dominant tree species in a forest area (located in Colorado, US), using the available features. The dataset may be found here https://archive.ics.uci.edu/dataset/31/covertype.

You may use any model you want and you are encouraged to compare many types of models, including simple benchmarks. Among the various methods explored, you must explore at least one type of neural network and one ensemble learning method.

Provide a short, general analysis of the dataset, that studies its statistical properties, outliers, correlation matrices, or any other interesting analysis. You may produce visualizations. If you present statistical analysis, discuss at least a little bit whether it is consistent with intuition or not (instead of for instance just showing a correlation matrix).

You must **compare** the various estimators / optimization procedures, from different points of views : scoring, training time, interpretability, robustness, size, or any point of view that you find interesting !

2.6 Clustering on the Fashion MNIST dataset

The Fashion MNIST dataset (https://github.com/zalandoresearch/fashion-mnist) contains 28×28 pictures of clothes, with labels (like "Dress", "Pullover", etc).

A labeled dataset is interesting to cluster, because we can then compare the clusters (learned in an unsupervised way) to the labels (that have not been used in training). This comparison can be done by using specific **scores**, like the Normalized Mutual Information (NMI) (https://scikit-learn.org/stable/modules/generated/sklearn.metrics.normalized_mutual_info_score.html), the Adjusted Rand Index (https://scikit-learn.org/stable/modules/generated/sklearn.metrics.adjusted_rand_score.html), and/or by producing and interpreting **visualizations**. Note that other scores also exist (https://scikit-learn.org/stable/api/sklearn.metrics.html).

Perform a clustering on the Fashion MNIST dataset. Discuss the important algorithmic details and research : for instance, the **metric** and optionally the **embedding(s)** and/or **dimensionality reduction** methods used. Again, it is very important to discuss and interpret the results obtained.

A ULTIMATE VOCAL REMOVER (UVR) IN MORE DETAILS

A.1 Installation

These steps are an example installation, I tested it on Ubuntu 24.04, with a x86_64 architecture.

— clone the repo (https://github.com/Anjok07/ultimatevocalremovergui/releases/tag/v5.6)

— sudo apt update && sudo apt upgrade

— sudo apt-get install -y ffmpeg python3-pip python3-tk

— add the uv environment files :

  - pyproject.toml
  - uv.lock
  - .python-version
- uv run UVR.py (requirements.txt will be deleted)

A.2 GUI and usage.

The GUI can be seen on figure 1. To start processing audio, you will need to download a model. While several model types are available in UVR, we will focus on MDX-Net models (but do not hesitate to explore the other ones if you are interested). After selecting MDX-Net in the "Choose process method", you can open the "Choose MDX-Net model" drop-down and open the model download center. There, you can for instance use "UVR-MDX-NET Inst HQ 3". The .onnx model will be stored in models/MDX_Net_Models/UVR-MDX-NET-Inst_HQ_3.onnx. After that, you can start processing audio.

A.3 Models

UVR allows to host several types of models.

VR architecture : CNN, the oldest model in the app. Mostly for separation in 2 stems. VR stands for "Vocal Removal".

MDX-Net : Mostly for separation in 2 stems. MDX stands for "Music Demixing".

demucs : Hybrid transformer, can be used for multi-stem separation. The name comes from "Deep extractor for music sources" and sounds like "demix".

A.4 Digital audio, inputs and outputs of the MDX-Net models (chunks and spectrograms)

Sounds we hear are continuous waveforms. But in the digital world, audio is **discretized**. Only a finite number of samples per second of the audio waveform are stored, typically 44100 samples per second. We note that under certain mathematical conditions, this is sufficient to perfectly reproduce the audio signal (Nyquist-Shannon theorem).

MDX-Net models are fed with **spectrograms** of audio **chunks**. A chunk is simply an audio piece, typically a small number of seconds (like 5.9). Primarily, the audio we hear is in the **time domain**. The signal is the discretized amplitude of the wave as a function of the time. The **Fourier transform** allows to compute which **frequencies** are represented in the signal. We note that the frequency spectrum is also discretized (here, typically we use 2048 frequency bins, stepped by 10.8 Hz). Here, spectrograms are a collection of Fourier transforms of audio **frames**. There are 256 frames in the chunk, each frame containing 23 ms of audio. We note that the frames **overlap**. You can find more information on this topic in the **my_models/src/spectrograms.py** file. In figures 2 and 3, we can see to example spectrograms.

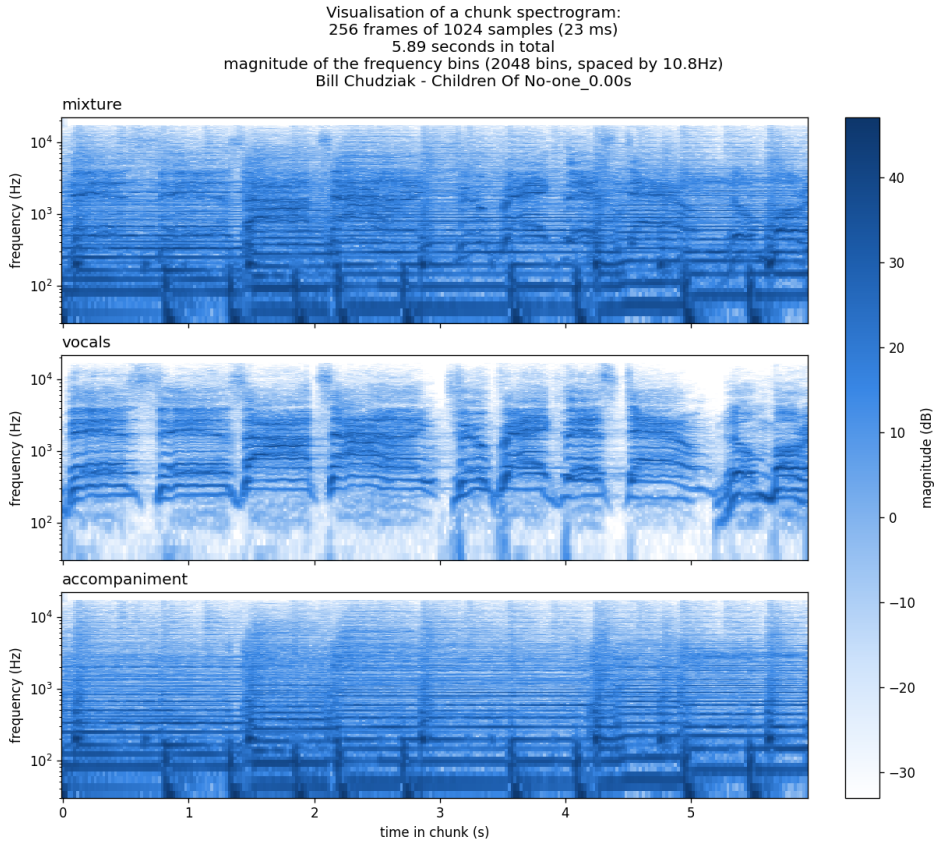


Figure 2 – Visualization of a chunk spectrogram from the MUSDB18 reduced dataset.

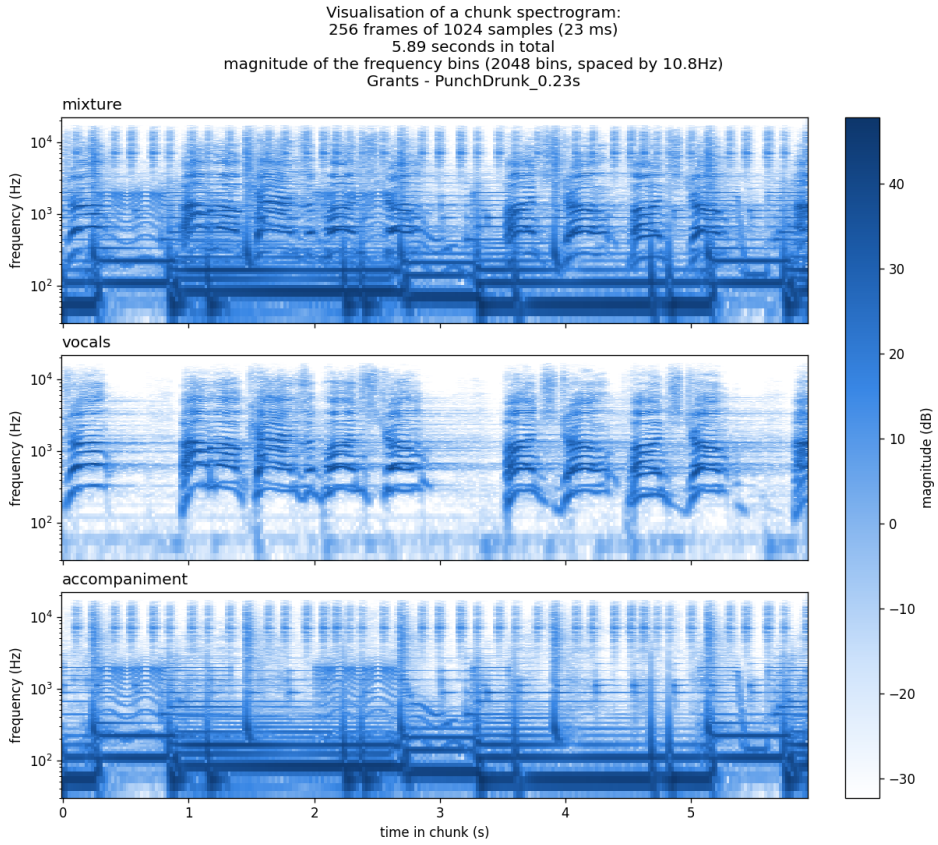


FIGURE 3 – Visualization of a chunk spectrogram from the MUSDB18 reduced dataset, from a different song

A.5 Model data and json files

MDX-Net models requires some metadata, stored as json files. These data contain necessary audio information, like the number of frequency bins in the spectrograms, or the number of samples in a frame.

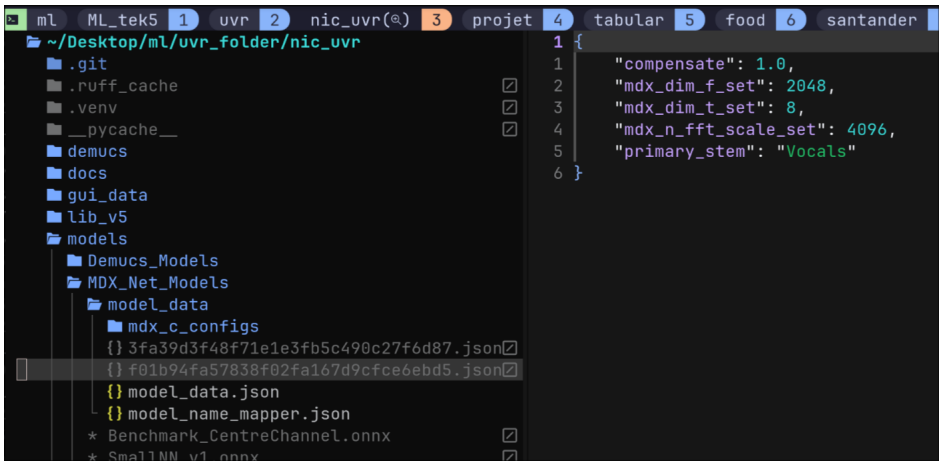


FIGURE 4 – An example model data json

A.6 MUSDB18 dataset

The MUSDB18 dataset (https://zenodo.org/record/3270814/files/MUSDB18-7-STEMS.zip) is an example small audio dataset containing both the full audio and the separated stems, that can thus be used for model training. It is provided to you here as a solution to make some quick tests, but you should explore other datasets, like for instance the full version, of other datasets.

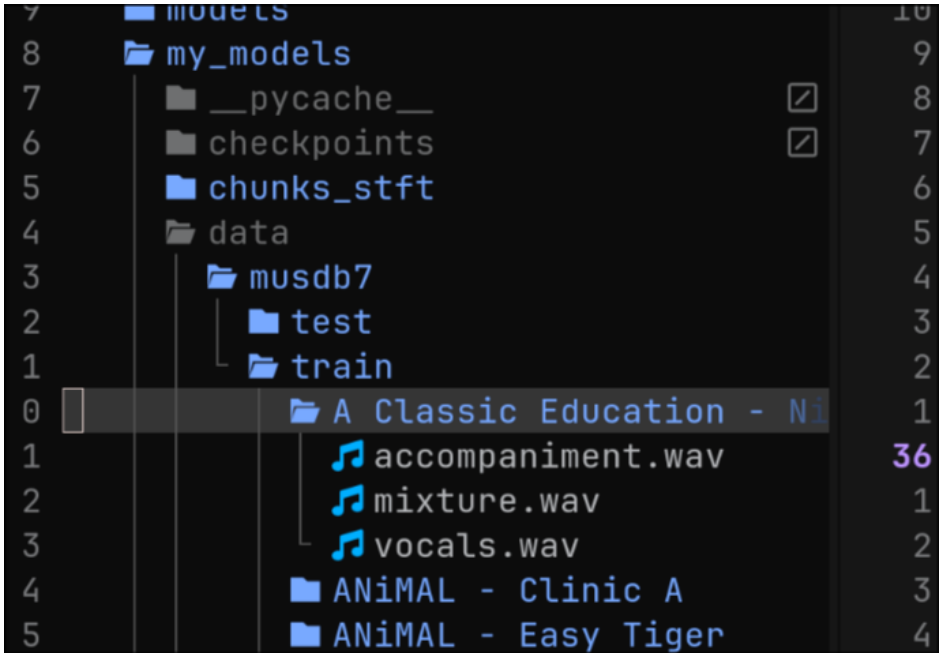


FIGURE 5 – In the dataset, for each song, the raw audio is provided, along with the 2 steps to separate.